Control Software Implementation

Overview

The Injera-making machine control software is a real-time embedded system built with FreeRTOS running on an STM32F446RE microcontroller. The system implements multi-threaded control of three subsystems: thermal control (heating), motion control (plate rotation), and dispense control (pump), with comprehensive safety monitoring.

Current baseline is aligned to `pancake-making-machine.ioc` (STM32CubeMX 6.15.0):

- Core clock source: HSI, `SYSCLK = 16 MHz`
- Time base: TIM6
- Enabled communication/control peripherals: SPI1, TIM1, TIM2, USART2
- GPIO labels use `ESTOP`, `COVER_SW`, `THETA_*`, `RADIAL_*`, `PWM_PUMP`, `BATTER_LOW`, `SPARE_DI`

Architecture

Task-Based Structure (FreeRTOS)

The control system runs 6 concurrent tasks with different priorities and execution rates:

Task	Priority	Rate	Function
Safety Monitor	5 (Highest)	1 kHz	E-stop, cover interlock, thermal cutoff monitoring
Thermal Control	4	25 Hz	PT100 sensor reading, PID loop, SSR duty calculation
Motion Control	4	25 Hz	Stepper motor velocity profiling, step pulse generation
Pump Control	3	10 Hz	PWM duty cycle adjustments, per-nozzle trim tuning
HMI	2	2 Hz	Serial command processing, status display, LED control
Logging	1 (Lowest)	1 Hz	Data logging to circular buffer

Safety-First Design: The Safety Monitor task runs at highest priority to ensure immediate response to safety-critical events (E-stop, cover open, overtemp).

Module Structure

```
firmware/stm32f446re/Core/
├── Inc/
│   ├── control_system.h       # Main control system header
│   ├── thermal_control.h      # Temperature PID control
│   ├── motion_control.h       # Stepper motor control
│   ├── pump_control.h         # PWM pump control
│   ├── safety_control.h       # Safety monitoring
│   ├── hmi.h                  # Human-machine interface
```

```
│   ├── logging.h              # Data logging
│   └── FreeRTOSConfig.h       # FreeRTOS configuration
└── Src/
    ├── main.c                 # FreeRTOS task creation & scheduling
    ├── control_system.c       # State machine & integration
    ├── thermal_control.c      # Thermal implementation
    ├── motion_control.c       # Motion implementation
    ├── pump_control.c         # Pump implementation
    ├── safety_control.c       # Safety implementation
    ├── hmi.c                  # HMI implementation
    └── logging.c              # Logging implementation
```

Key Features

1. Thermal Control (220°C, ±3°C)

PID Controller Implementation:

- PT100 RTD sensor → MAX31865 (SPI) → Temperature reading
- Proportional-Integral-Derivative feedback control
- Anti-windup integral term to prevent saturation
- 25 Hz sampling rate; 10 Hz SSR drive frequency
- Temperature stabilizes to setpoint within ±3°C

Callendar-Van Dusen RTD Equation:

```
R(T) = R0[1 + A*T + B*T²]  (for T ≥ 0°C)
```

PID Tuning Parameters (starting values):

- Kp = 0.8 (proportional gain)
- Ki = 0.15 (integral gain)
- Kd = 0.2 (derivative gain)

2. Motion Control (0.2 rev/s rotation)

Stepper Motor Interface:

- NEMA23 motor with 0.9° step angle
- TB6600 microstepping driver (4x microsteps, 1600 steps/rev)
- Step/Direction/Enable logic

Features:

- Homing sequence via home switch
- Smooth acceleration/deceleration ramps
- Position and speed tracking
- Fault detection (stall prevention)

3. Pump Control (28.3 mL/s total, 7 nozzles)

PWM-Based Pump Control:

- PWM output on `PB6 (PWM_PUMP)` via TIM4 CH1 in firmware
- Current runtime configuration: 10 Hz PWM (aligned with low-rate dispense control loop)
- Dispensing: 5 seconds per cycle (141.5 mL total)
- Per-nozzle trim adjustments (0-100%)

Flow Distribution by Ring (mL/s):

```
Ring 1 (center):  0.8  mL/s
Ring 2:           1.2  mL/s
Ring 3:           1.6  mL/s
Ring 4:           2.0  mL/s
Ring 5:           2.4  mL/s
Ring 6:           2.8  mL/s
Ring 7 (outer):   18.7 mL/s (remainder)
Total:            28.3 mL/s
```

4. Safety Monitoring (1 kHz hardwired monitoring)

Safety Inputs (Active Low):

- **E-STOP:** Dual NC channels (hardware safety chain)
- **COVER INTERLOCK:** Door/cover switch (NC)
- **THERMAL CUTOFF:** Device at 250°C threshold

Debouncing:

- E-Stop: 50 ms debounce
- Cover: 100 ms debounce

Immediate Actions on Fault:

- Disable SSR heater
- Disable stepper motor
- Disable pump PWM
- Set system to ESTOP mode

5. State Machine

```
IDLE ──[warmup]──> WARMUP ──[stable]──> READY
                                          ↓
                                      DISPENSE
                                          ↓
                                      COOLDOWN ──[repeat]──┐
                                                           │
```

```
CIP <──────────[cip command]──────────────────────────────┘

ERROR ──[manual reset]──> IDLE

ESTOP ──[watchdog/reset]──> ERROR
```

State Details:

- **IDLE:** Waiting for operator command
- **WARMUP:** Heating plate to setpoint (210-230°C)
- **READY:** Plate at stable temperature, ready to dispense
- **DISPENSE:** Active 5-second cycle (rotate, pump, track)
- **COOLDOWN:** Safe cooldown before handling (< 100°C)
- **CIP:** Cleaning-in-place mode (pump only, no heat)
- **ERROR:** Fault occurred, manual acknowledge required
- **ESTOP:** Emergency stop active, full shutdown

Communication Interface

UART Serial Protocol (115200 baud, 8N1)

Command Format: Single character commands sent via USART2

Command	Function	Example
S	Start cycle	S
H	Home theta axis	H
Txxx	Set temperature	T220 (220°C)
Pxx	Set pump duty	P75 (75%)
C	Start CIP mode	C
E	Emergency stop	E
Q	Query status	Q
L	Dump logs	L
A	Show statistics	A
?	Help menu	?

Status Output Example:

```
=== System Status ===
Mode: READY
Temp: 220.5 C (setpoint: 220.0 C)
Pump: OFF (duty: 0%)
Theta: OFF (RPM: 0.0)
```

```
E-Stop: OK, Cover: CLOSED
Cycles: 42
```

Data Logging

Circular Buffer: 1000 entries at 1 Hz (16.7 minutes of continuous logging)

Log Entry Structure:

```
typedef struct {
    uint32_t timestamp_ms;           // System uptime in ms
    float plate_temperature_c;       // Current temperature
    float pump_duty_percent;         // Pump PWM duty
    float theta_rpm;                 // Motor RPM
    float theta_position_revs;       // Rotational position
    bool estop_active;               // E-stop state
    bool cover_open;                 // Door state
    bool pump_enabled;               // Pump status
    bool motion_enabled;             // Motor status
    uint8_t machine_mode;            // Current state
    uint8_t fault_code;              // Fault flags
    uint16_t cycle_number;           // Cycle counter
} log_entry_t;
```

Hardware Interface

GPIO Assignments (from main.h)

```
**Port A (GPIOA):**
PA0  - ESTOP (input, active low)
PA1  - COVER_SW (input, active low)
PA2  - UART_TX
PA3  - UART_RX
PA5  - SPI1_CLK (MAX31865)
PA6  - SPI1_MISO
PA7  - SPI1_MOSI
PA8  - RADIAL_STEP (TIM1_CH1)

**Port B (GPIOB):**
PB3  - THETA_STEP (TIM2_CH2)
PB4  - THETA_DIR (stepper direction)
PB5  - THETA_EN (stepper enable)
PB6  - PWM_PUMP (pump PWM output)
PB7  - Power_Indicator (LED)
PB8  - RADIAL_DIR
PB9  - RADIAL_EN

**Port C (GPIOC):**
```

```
PC4  - START_BTN
PC5  - STOP_BTN
PC6  - HOME_THETA
PC7  - HOME_RADIAL
PC10 - BATTER_LOW (input)
PC11 - SPARE_DI

**SPI1:**
SPI1_MOSI - PA7
SPI1_MISO - PA6
SPI1_CLK  - PA5

**Timers:**
TIM1 CH1 - PA8 output (used by thermal SSR control path)
TIM2 CH2 - PB3 stepper pulse generation
TIM4 CH1 - PB6 pump PWM output (initialized in firmware user code)
```

Compilation and Integration

Prerequisites

- STM32CubeMX project with basic initialization
- FreeRTOS kernel included in project
- ST HAL drivers
- C99 or later compiler

Files to Build

```
Core/Src/main.c
Core/Src/control_system.c
Core/Src/thermal_control.c
Core/Src/motion_control.c
Core/Src/pump_control.c
Core/Src/safety_control.c
Core/Src/hmi.c
Core/Src/logging.c
```

Build Configuration

```
# Example CMake configuration
target_include_directories(firmware
    PRIVATE Core/Inc
    PRIVATE Drivers/...
    PRIVATE FreeRTOS/include
)

target_sources(firmware PRIVATE
```

```
    Core/Src/main.c
    Core/Src/control_system.c
    Core/Src/thermal_control.c
    # ... other sources
    FreeRTOS/tasks.c
    FreeRTOS/queue.c
    FreeRTOS/list.c
    FreeRTOS/port.c
    # ... other FreeRTOS files
)
```

Tuning Guide

Thermal PID Tuning

1. **Manual Mode Testing:**

   ```
   Command: T220  (set to 220°C)
   Command: Q     (monitor temperature rise)
   ```

2. **Initial Gains** (conservative):
   - Kp = 0.8, Ki = 0.15, Kd = 0.2
3. **Adjust if Needed:**
   - **Overshoots setpoint:** Reduce Kp or increase Kd
   - **Slow response:** Increase Kp or Ki
   - **Oscillation:** Increase Kd
   - **Offset from setpoint:** Increase Ki (integral term)

Pump Flow Tuning

1. Determine total flow rate with test fluid
2. Calculate individual nozzle flows for each ring
3. Adjust trim values using serial commands
4. Re-measure and refine

Example:

```
Command: P75    (set pump duty to 75%)
Command: Q      (check status; measure actual flow rates)
Command: T7,50  (trim nozzle 7 to 50%)
```

Safety Considerations

Thermal Safety

- PT100 sensor failure → SSR disabled
- Temperature > 250°C → Thermal cutoff triggers
- Improper reading → Conservative disable

Motion Safety

- Homing required before dispense
- E-stop drives `THETA_EN` disable path in firmware and shuts motion task outputs
- Cover interlock prevents operation when open

System Faults

- Sensor loss
- Motor stall
- Pump pressure loss (future)
- Watchdog timeout
- Over-current (future)

Performance Targets

Metric	Target	Status
Cycle time	≤ 5.0 s	✓ Design goal
Temperature stability	±3°C	✓ PID tuned
Flow accuracy	±5% total, ±10% per ring	✓ Trim calibration
Thickness uniformity	±0.3 mm	✓ Hardware dependent
Safety response	< 50 ms	✓ 1 kHz monitoring
Uptime	≥ 95% per 8h	Target

Testing Checklist

- ☐ Compile without errors/warnings
- ☐ Flash and verify startup messages on UART
- ☐ Test E-stop response (< 50 ms shutdown)
- ☐ Test cover interlock (prevent heat/motion)
- ☐ Warm-up cycle (reach 220°C ±3°C)
- ☐ Dispense cycle (5.0 s ±0.2 s)
- ☐ Motor homing and rotation
- ☐ Pump PWM output and trim adjustment
- ☐ UART serial commands
- ☐ Data logging 1 Hz
- ☐ Cooldown safety
- ☐ CIP mode operation

Modifications for Deployment

1. **Configure PID gains** in thermal_control.c based on plate thermal mass
2. **Adjust motor speed ramps** in motion_control.c for smooth operation
3. **Calibrate flow targets** in pump_control.c per manifold design
4. **Tune debounce timers** in safety_control.c if needed
5. **Enable statistics logging** if telemetry required
6. **Add watchdog monitoring** if extreme reliability needed

Future Enhancements

- [ ] Flow sensor feedback (adaptive pump control)
- [ ] Vision-based quality checking
- [ ] Cloud telemetry and OTA updates
- [ ] Multi-unit networked operation
- [ ] Advanced diagnostics and self-test
- [ ] Predictive maintenance alerts
- [ ] Recipe management system

Version: 1.1
Last Updated: 2026-08-07
Target Hardware: STM32F446RE with FreeRTOS